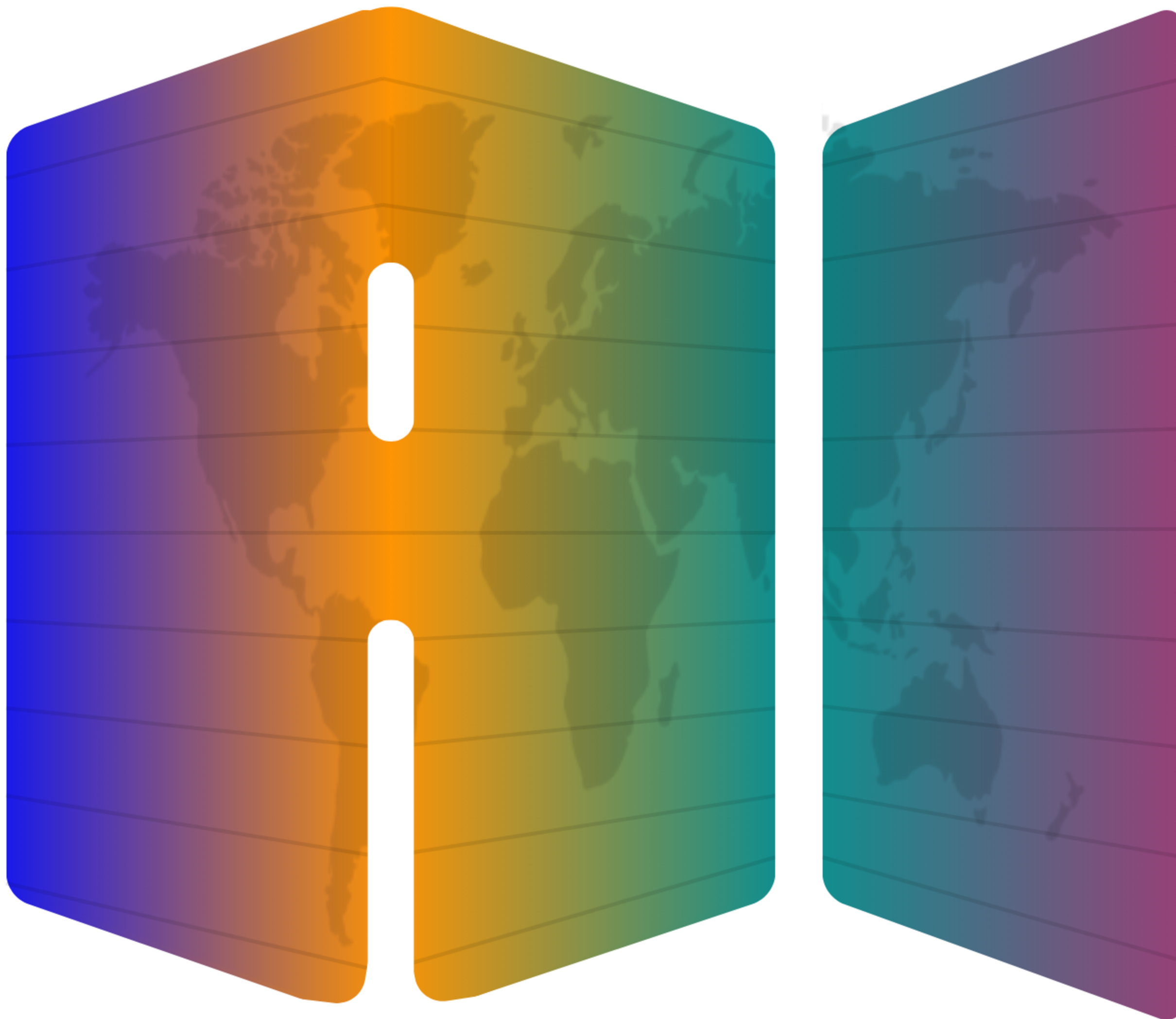
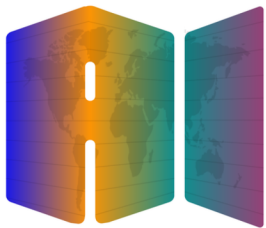




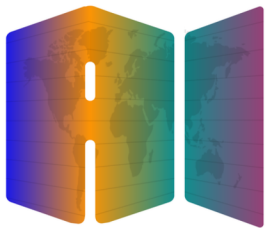
Model Context Protocol

Lectures by Elie Schoppik



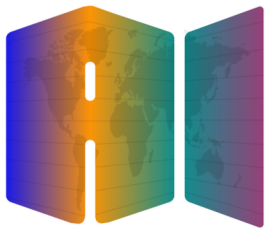
Model Context Protocol

Why MCP



Why MCP

Models are only as good as the
context given to them



Why MCP

What is the Model Context Protocol (MCP)

MCP is an open protocol that standardizes how your **LLM applications** connect to and work with your **tools & data sources**.

REST APIs

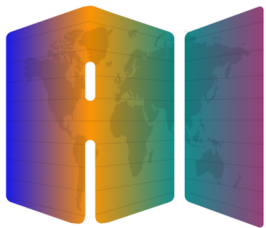
Standardize how **web applications** interact with the **backend**

LSP

Standardizes how **IDEs** interact with **language-specific tools**

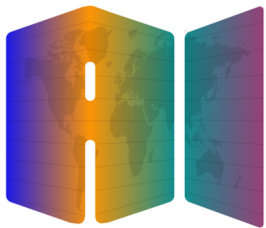
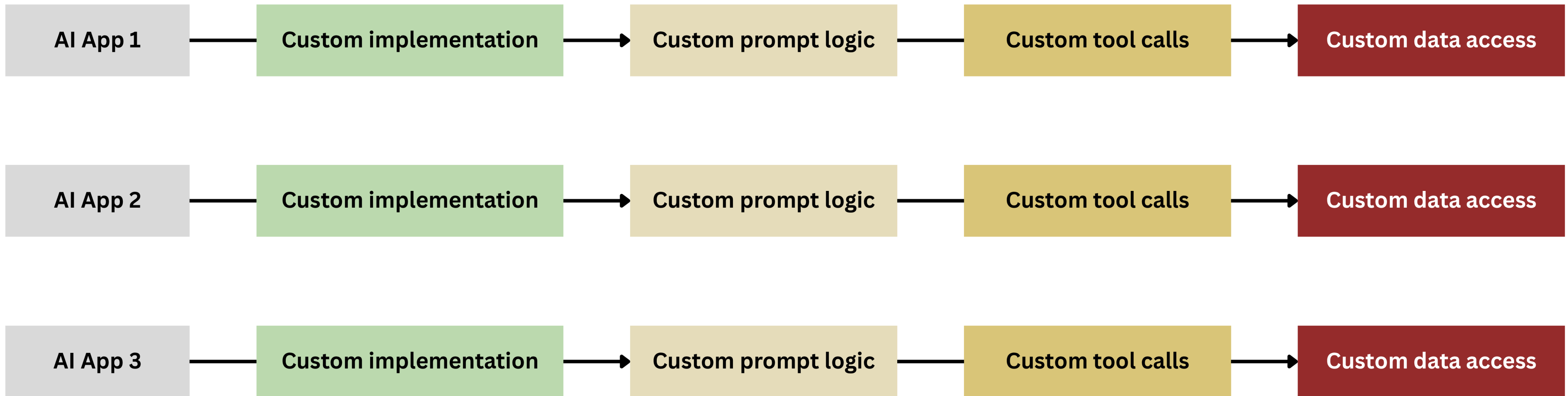
MCP

Standardizes how **AI applications** interact with **external systems**



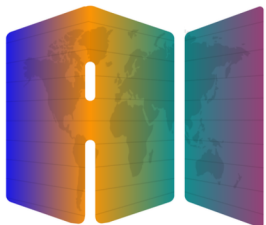
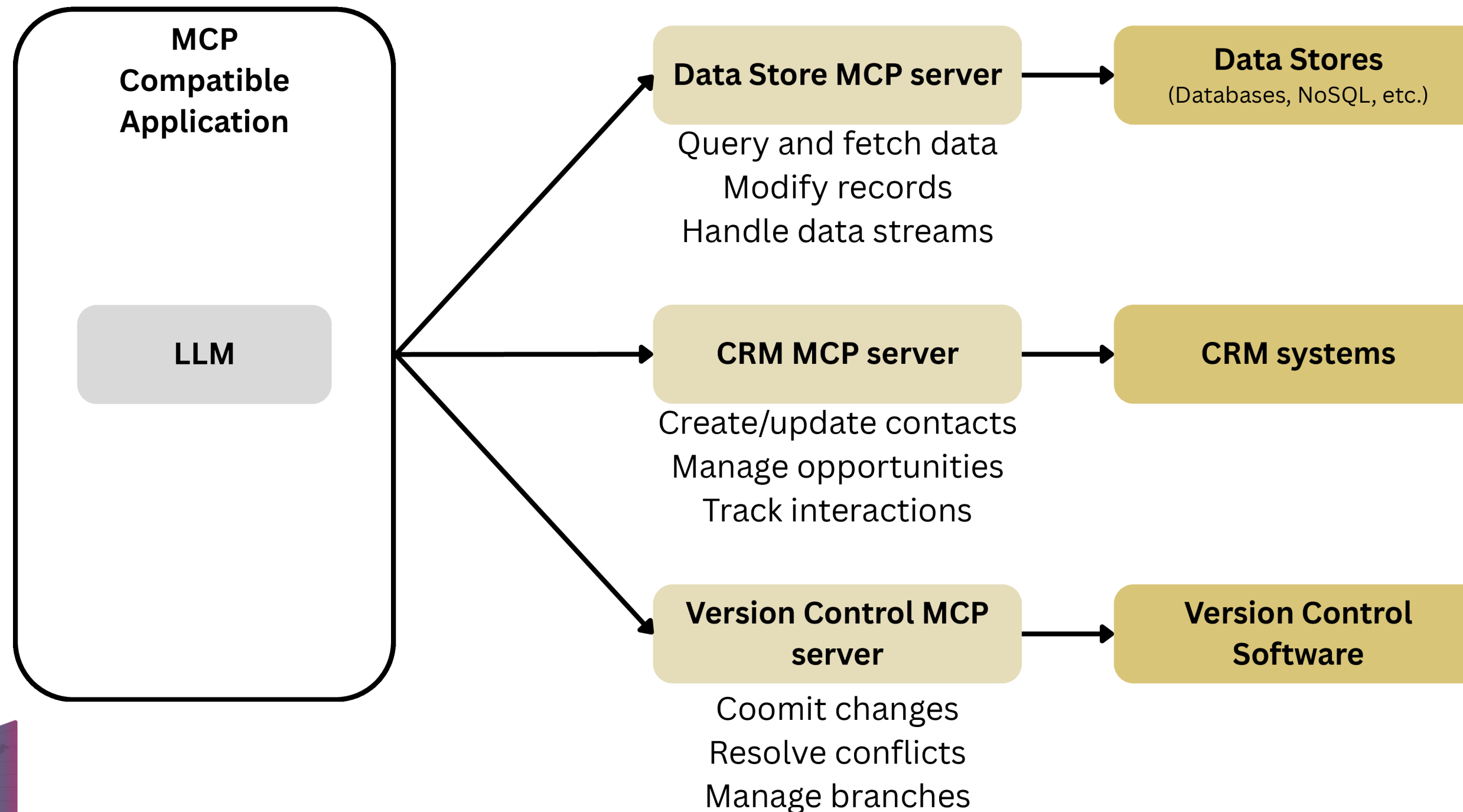
Why MCP

Without MCP: Fragmented AI Development



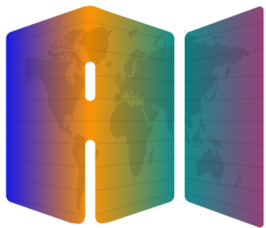
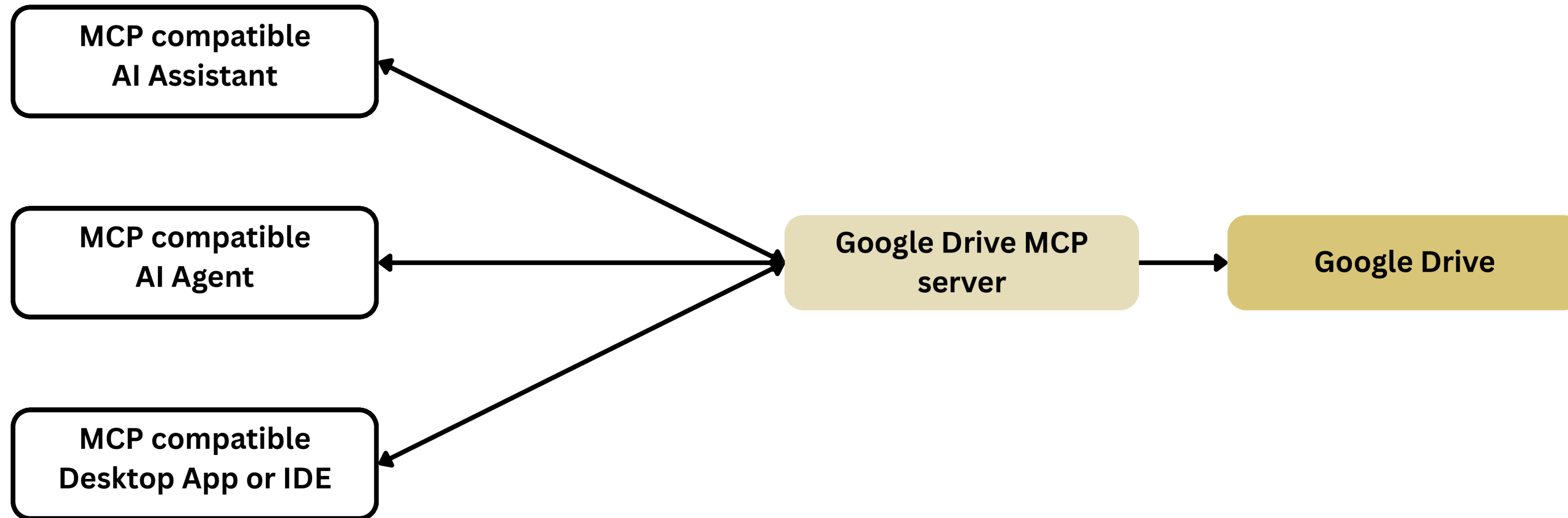
Why MCP

With MCP: Standardized AI Development



Why MCP

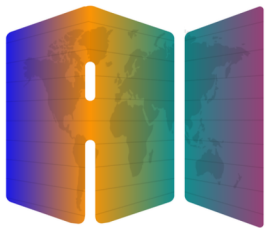
With MCP: MCP servers are reusable by various AI applications



Why MCP

With MCP: Standardized AI Development

- **For AI application developers**
Connect your app to any MCP server with 0 additional work
- **For tool or API developers**
Build an MCP server once which can be adopted everywhere
- **For AI application users**
AI applications have extensive capabilities
- **For enterprises**
Clear separation of concerns between AI product teams



Why MCP

The MCP Ecosystem is Growing Fast

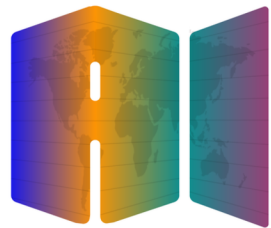
AI Applications, from IDEs to Agents

Server Builders (3000+ and counting)

Enterprises, for internal and external use cases

Open Source Contributors

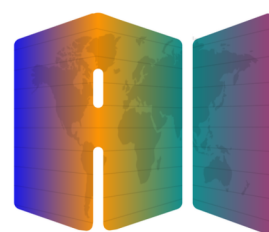
And a lot more...



Why MCP

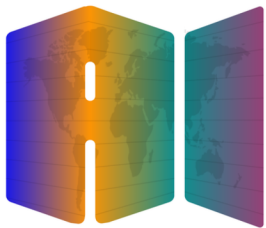
Common Questions

Who authors the MCP Server?	Anyone! Often the service provider itself will make their own MCP implementation. You can make an MCP server to wrap up access to some service.
How is using an MCP Server different from just calling a service's API directly?	MCP servers provide tool schemas + functions. If you want to directly call an API, you'll be authoring those on your own
Sounds like MCP Servers and tool use are the same thing	MCP Servers provide tool schemas + functions already defined for you



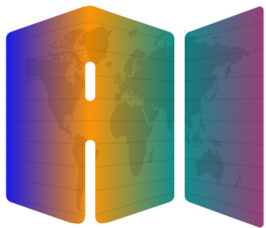
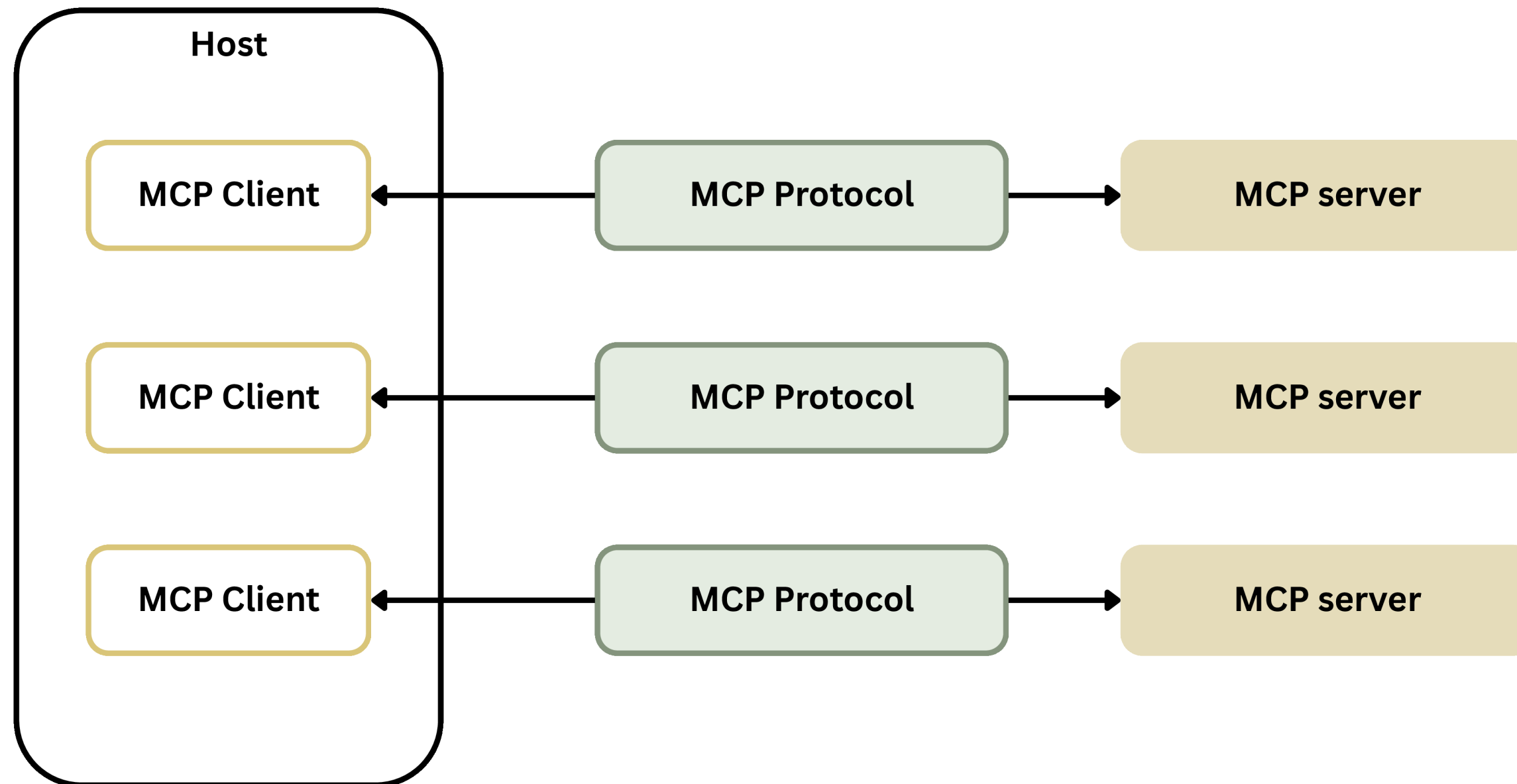
Model Context Protocol

MCP Architecture



MCP Architecture

Client-Server Architecture



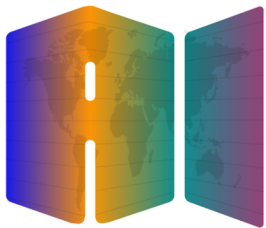
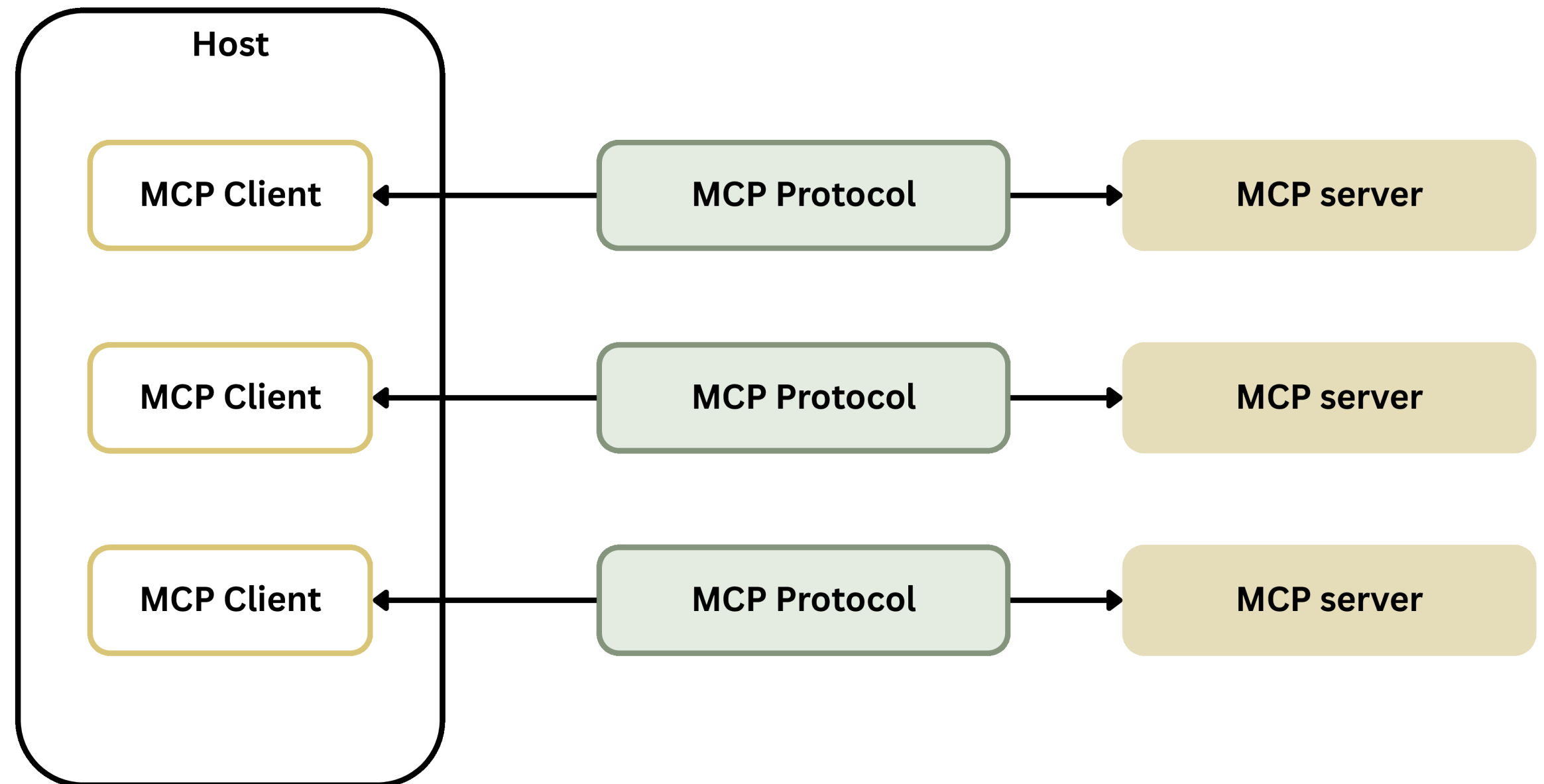
MCP Architecture

Client-Server Architecture

Hosts are LLM applications that want to access data through MCP (ex: Claude Desktops, IDEs, AI agents).

MCP Servers are lightweight programs that each expose specific capabilities through MCP.

MCP Clients maintain 1:1 connections with servers, inside the host application



MCP Architecture

How does it work?

MCP Client

Invokes **Tools**
Queries for **Resources**
Interpolates **Prompts**

MCP server

Exposes **Tools**
Exposes **Resources**
Exposes **Prompt Templates**

Tools

Functions and tools that
can be invoked by the
client

Retrieve / search

Send a message

Update DB records

Resources

Read-only data or
exposed by the server

Files

Database Records

API Responses

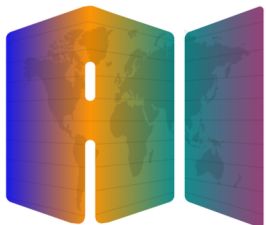
Prompt Templates

Pre-defined templates
for AI interactions

Document Q&A

Transcript Summary

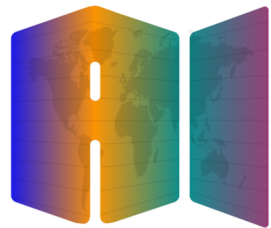
Output as JSON



MCP Architecture

Defining a Tool

- MCP provides SDK's for building servers and clients in a variety of languages
- We will use the Python MCP SDK
- The Python SDK makes it very easy to declare tools



MCP server

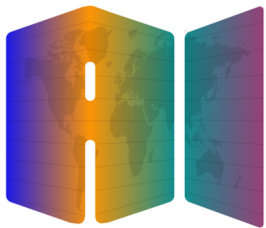
```
@mcp.tool()
def add(a: int, b: int) -> int:
    """
    Args:
        a: First number to add
        b: Second number to add

    Returns:
        The sum of the two numbers
    """
    return a + b
```

MCP Architecture

Resources

- Allow the MCP Server to expose data to the client
- Similar to GET request handlers in a typical HTTP server
- Can return any type of data - strings, JSON, binary, etc.
 - You set the `mime\_type` to give the client a hint as to what data you are returning
- Two types: direct and templated



MCP server

Direct

```
@mcp.resource(  
    "docs://documents",  
    mime_type="application/json"  
)  
def list_docs():  
    # Return a list of document names
```

Templated

```
@mcp.resource(  
    "docs://documents/{doc_id}",  
    mime_type="text/plain"  
)  
def fetch_doc(doc_id: str):  
    # Return the contents of a doc
```

MCP Architecture

Direct
resources

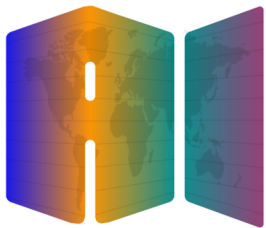
```
> Can you please summarize the contents  
of @  
deposition.md Resource  
design.md Resource  
financials.md Resource  
outlook.md Resource  
plan.md Resource  
spec.md Resource
```

...You need the
MCP Server to
give a *list* of
documents

Templated
resources

```
Answer the users query:  
<query>  
What's in the @report.pdf file?  
</query>  
The user may have referenced a document.  
Here is it's contents:  
<document id="report.pdf">  
...condition of a 20m condenser tower...  
</document>
```

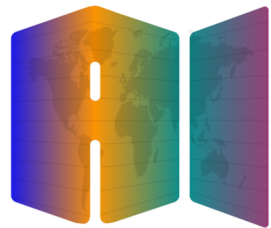
...You need the
MCP Server to
give the
contents of a
single document



MCP Architecture

Prompts

- Defines a set of User and Assistant messages that can be used by the client
- These prompts should be high quality, and well-tested.



MCP server

Prompt

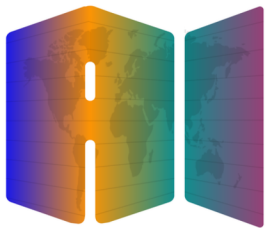
```
@mcp.prompt(  
    name= "format",  
    description= "Rewrites the  
contents of a document in Markdown  
format",  
)  
def format_document(  
    doc_id: str,  
) → list[base.Message]:  
    # Return a list of messages
```

MCP Architecture

If you left this process up to a user,
here's what they might write:

Convert report.pdf to markdown

Yes, it'd work, but the user
might get a better result with
some strong prompt
engineering



They have more luck if they use a
thoroughly evaluated prompt instead!

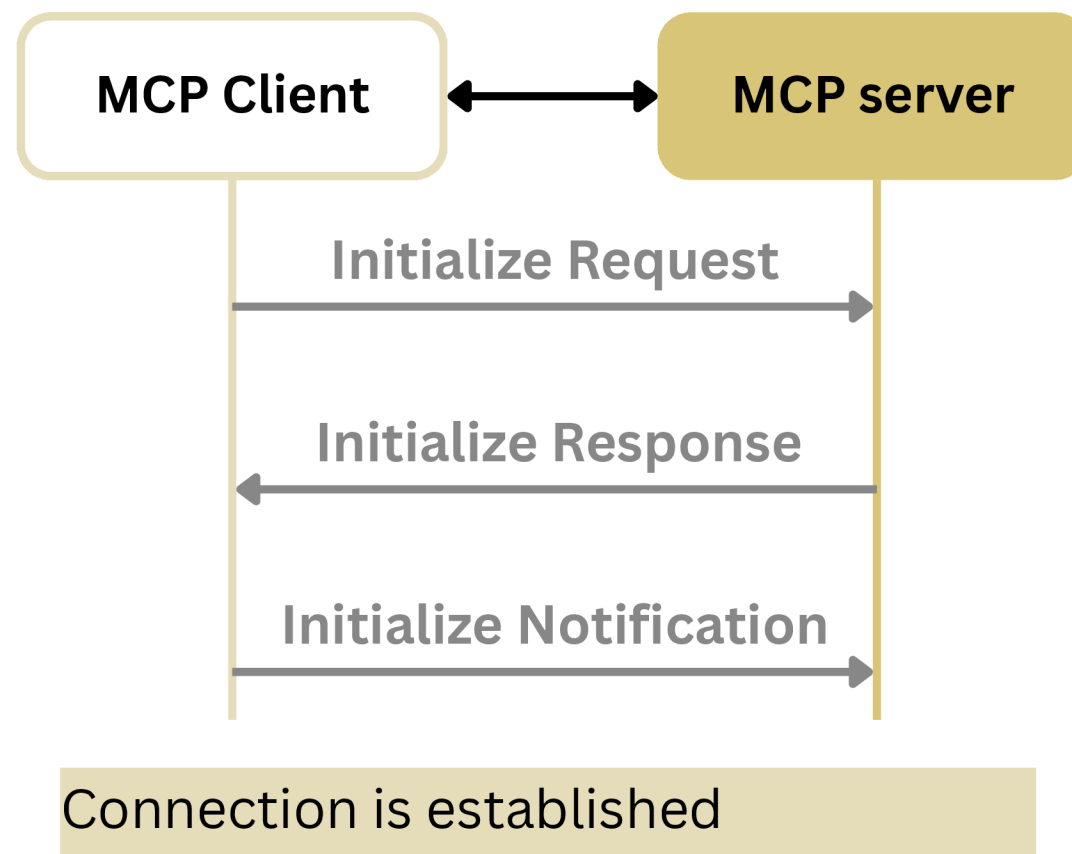
```
You are a document conversion specialist tasked with rewriting documents in Markdown
format. Your goal is to take the content of a given document and convert it into
well-structured Markdown, preserving the original meaning and enhancing readability.
Here is the identifier of the document you need to convert:
<document_id>
{{doc_id}}
</document_id>
Instructions:
1. Retrieve the content of the document associated with the given document_id.
2. Analyze the structure and content of the document.
3. Convert the document to Markdown format, following these guidelines:
- Use appropriate header levels (# for main titles, ## for subtitles, etc.)
- Properly format lists (both ordered and unordered)
- Use emphasis (*italic* or **bold**) where appropriate
- Add links and images using Markdown syntax if present in the original document
- Preserve any special formatting or structure that's important to the document's
meaning
Before providing the final Markdown output, in <document analysis> tags:
- Identify the main sections and subsections of the document
- Count the number of sections and subsections to ensure proper nesting of headers
This will help ensure a thorough and well-organized conversion.
After your analysis, present the converted document in Markdown format. Use ``` markers
to denote the beginning and end of the Markdown content.
Example output structure:
<document_analysis>
[Your analysis of the document structure and conversion plan]
</document_analysis>
```markdown
Document Title
Section 1
Content of section 1...
Section 2
Content of section 2...
- List item 1
- List item 2
[Link text](https://example.com)
![[Image description]](image-url.jpg)
...

Please proceed with your analysis and conversion of the document.
```

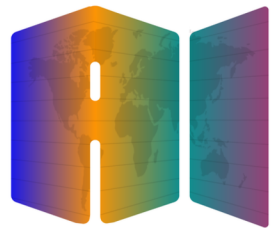
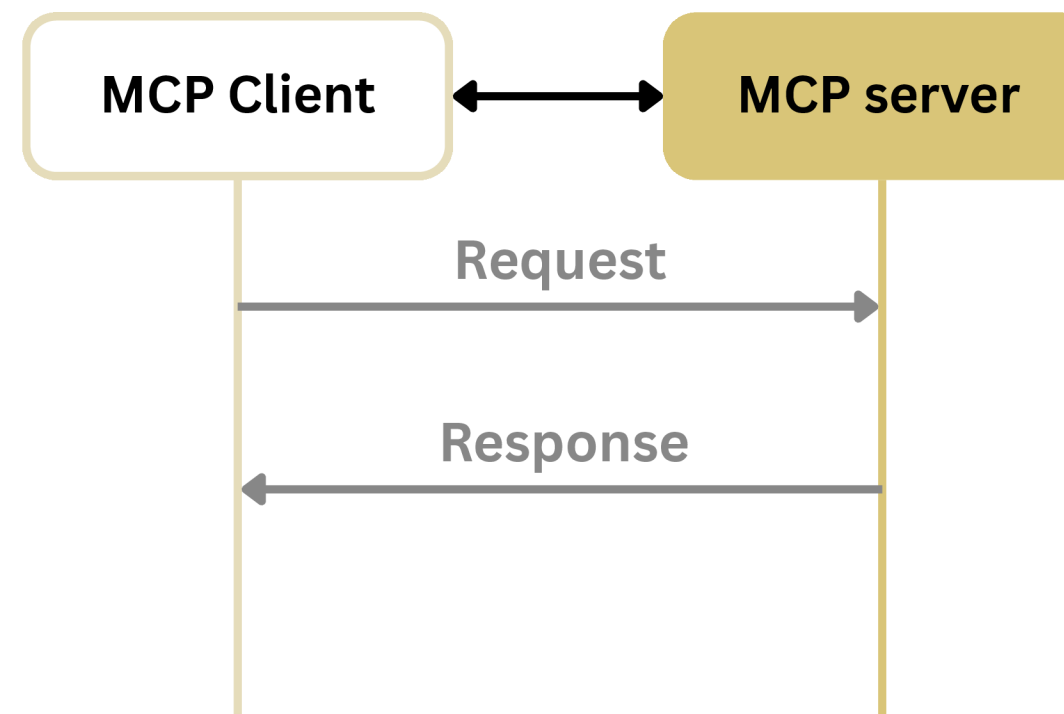
# MCP Architecture

## Communication Lifecycle

### 1. Initialization



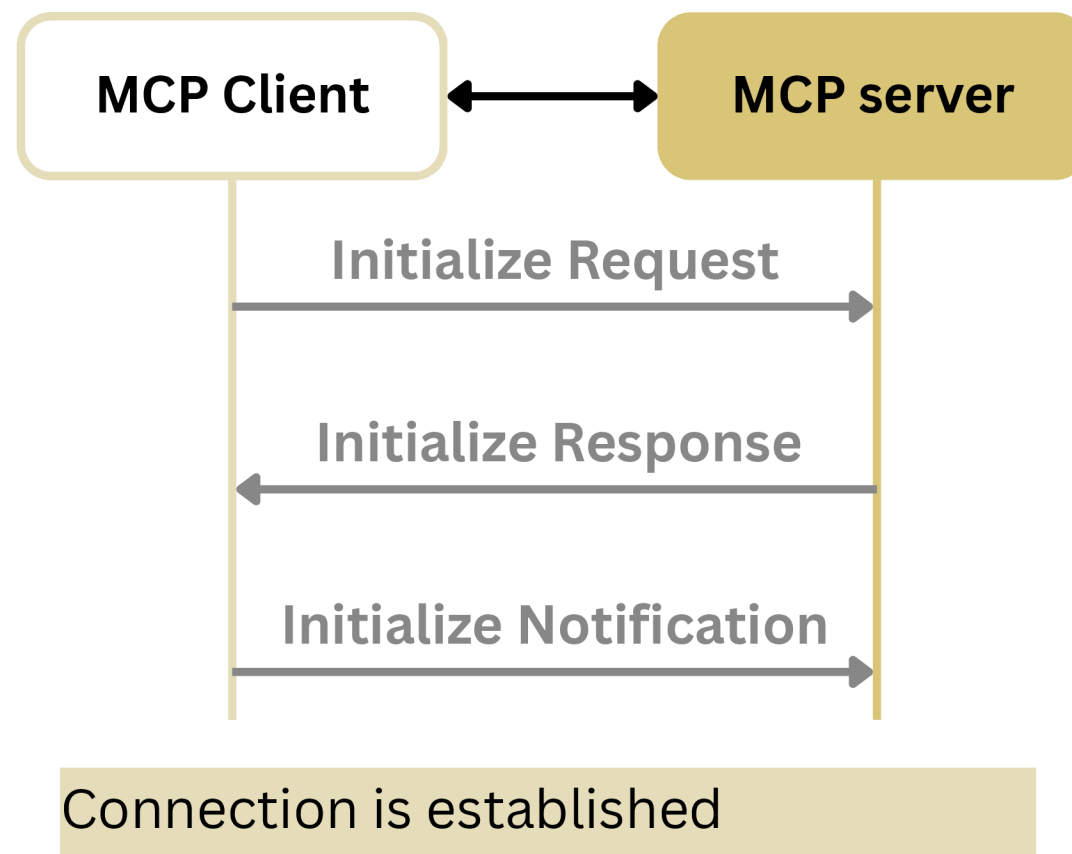
### 2. Message Exchange



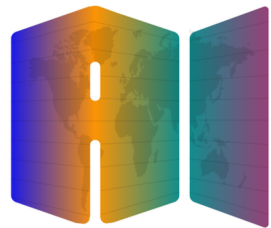
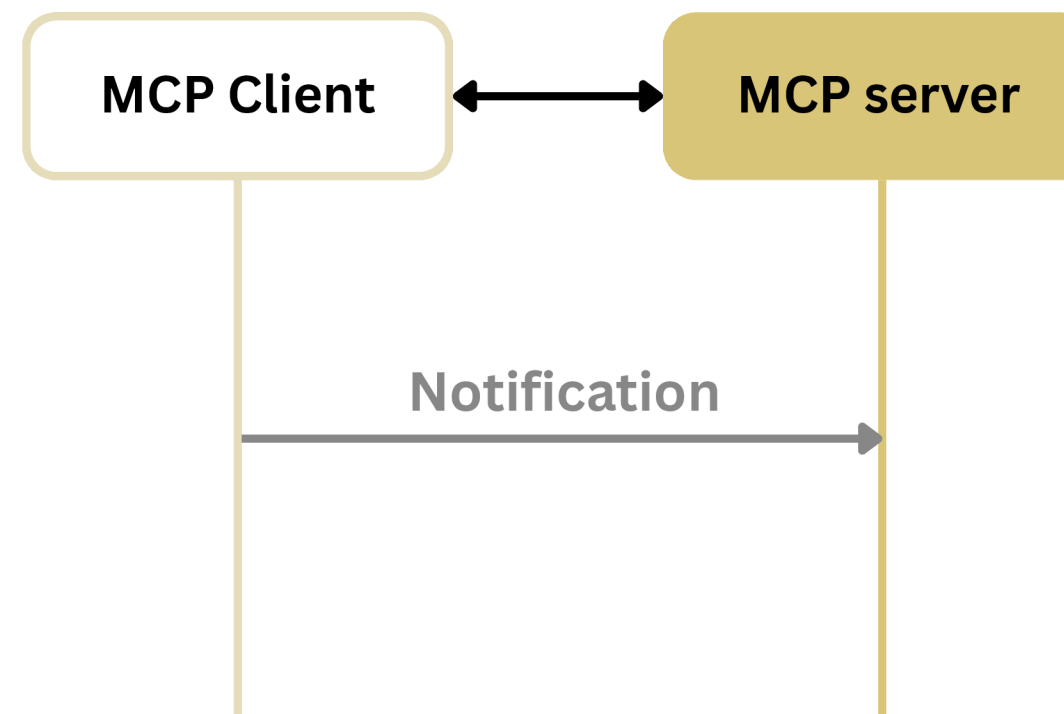
# MCP Architecture

## Communication Lifecycle

### 1. Initialization



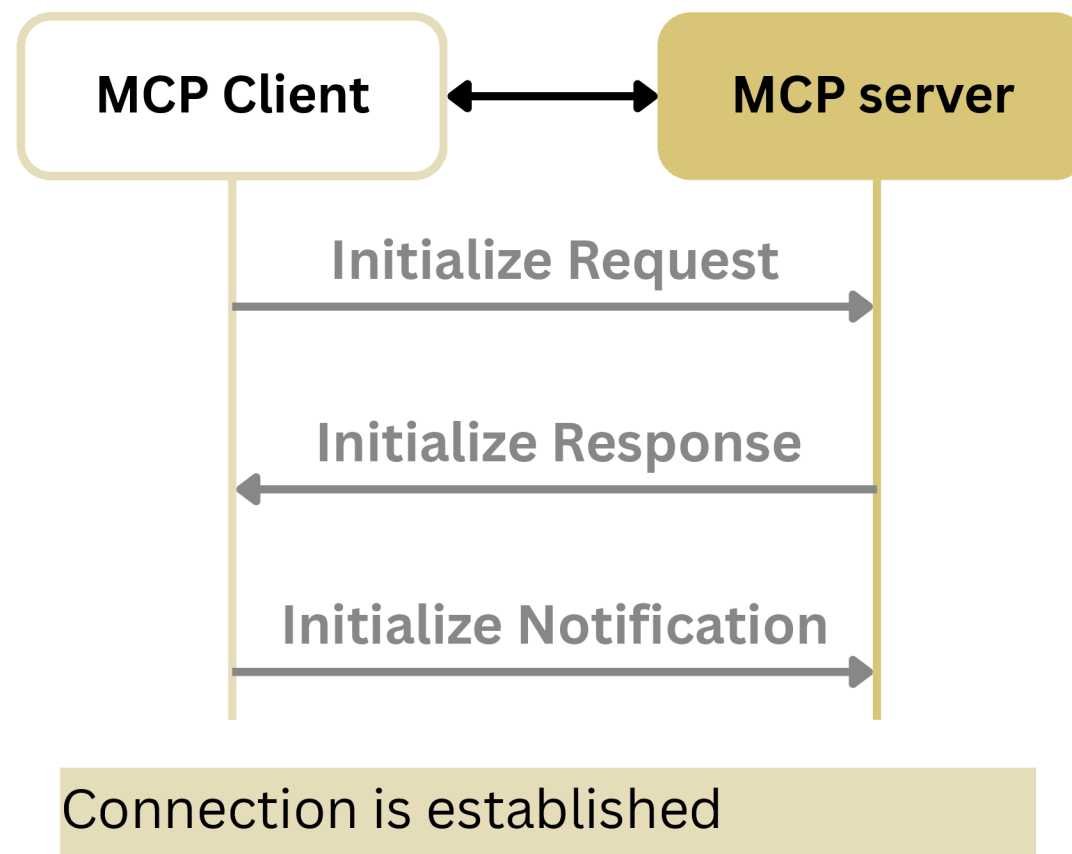
### 2. Message Exchange



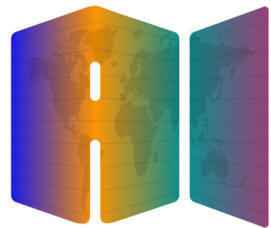
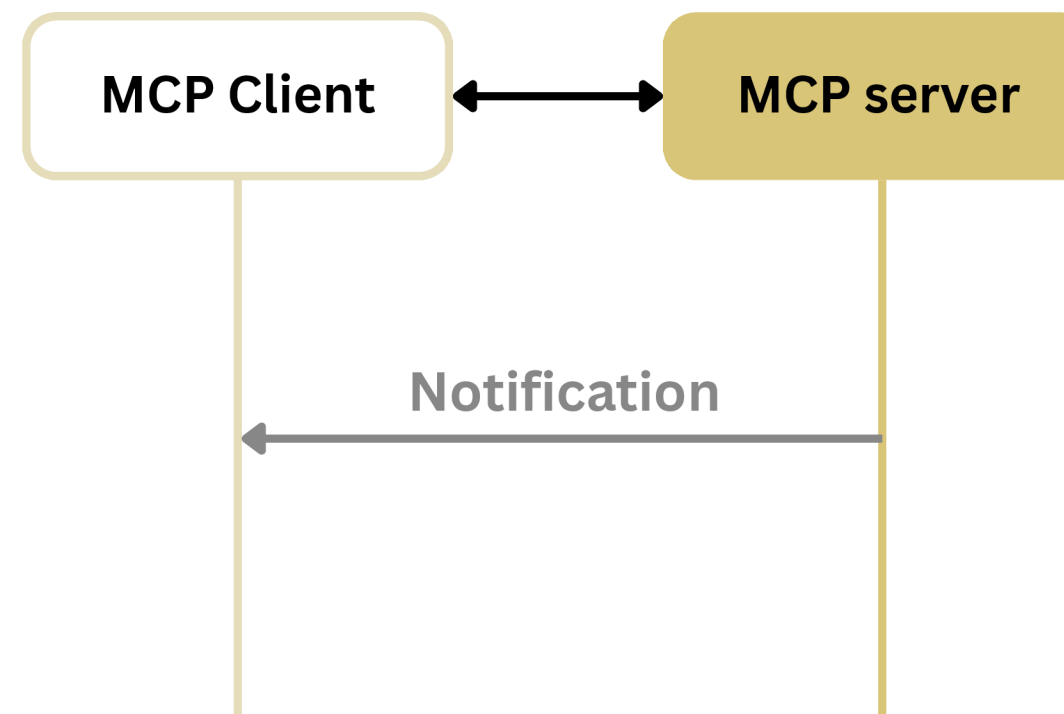
# MCP Architecture

## Communication Lifecycle

### 1. Initialization



### 2. Message Exchange

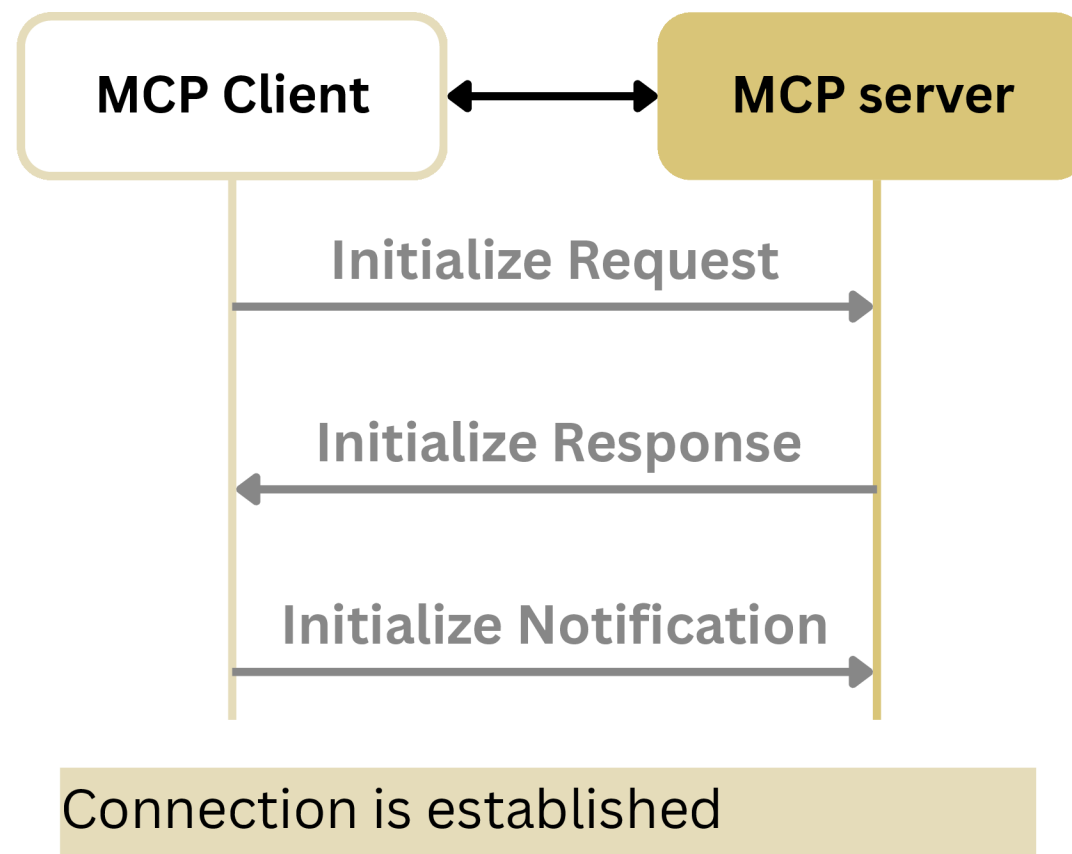




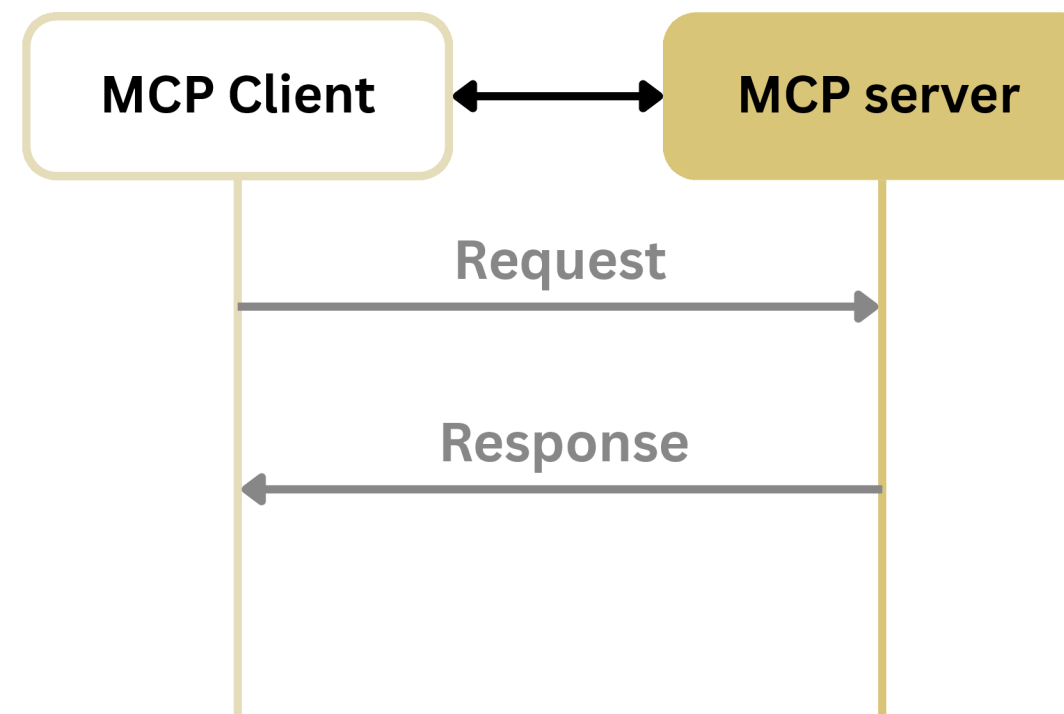
# MCP Architecture

## Communication Lifecycle

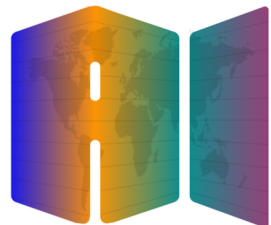
### 1. Initialization



### 2. Message Exchange



### 3. Termination

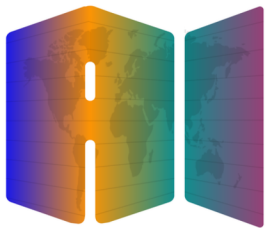


# MCP Architecture

## MCP Transports

A transport handles the underlying mechanics of how messages are sent and received between the client and server.

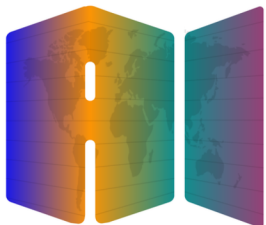
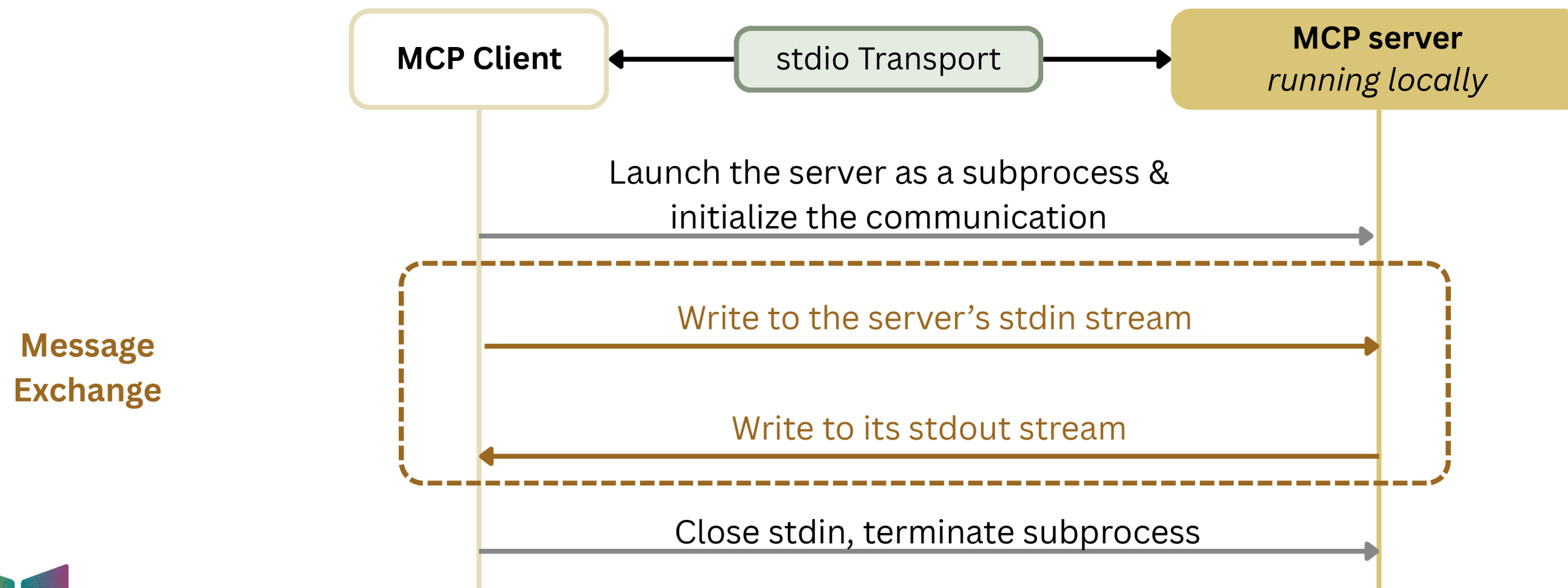
1. For servers running locally: **stdio** (standard input output)
2. For remote servers:
  - a. **HTTP+SSE (Server Sent Events)**
  - b. **Streamable HTTP**



# MCP Architecture

## Standard IO (stdio) Transport

When running servers locally, stdio is most commonly used

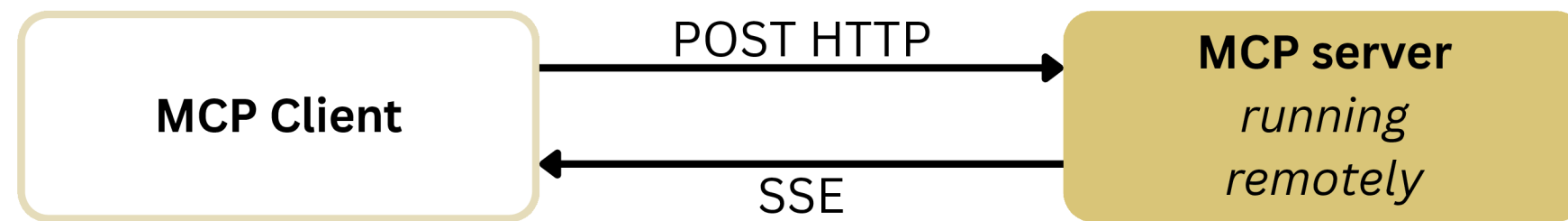




# MCP Architecture

## Transports for Remote Servers

### HTTP + SSE

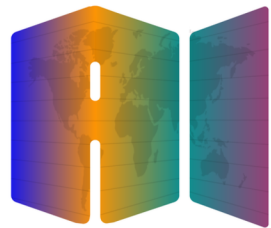


### Streamable HTTP



### Stateful Connection

### Allow for Stateless or Stateful Connection



# MCP Architecture

## Streamable HTTP Transport

For remote MCP servers the Streamable HTTP transport is used. This supports stateless connections as well as stateful connections with the ability to opt into Server Sent Events

